

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\agent-ac25f702a48f8d644.jsonl`  
- SHA-256: `f06b874f0855a20f3c2398abdc45953e2acc9e76b96b314b523d53959bf5355`  
- Source modified: `2026-06-22T17:45:37+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T17:42:09.659Z)

Adversarial code review of a change in the git worktree at `C:\Users\avidu\Projects\bhi-asyncnccompile` (a FastAPI badminton rally-detection engine). Find REAL bugs only ? correctness, security, backward-compat/default-OFF regressions, race conditions, or schema-migration errors. No style nits, no speculative refactors. Be skeptical and try to break it.

The change makes `POST /api/compile` **asynchronous** (issue #329): it used to run the ffmpeg reel stitch synchronously (which 524'd behind Cloudflare's ~100s edge timeout on slow CPU stitches). Now it returns **202** + a compile job id**, runs the stitch on the background executor (mirroring `/api/process`), and the client polls `GET /api/jobs/{id}`. The jobs table gained `kind` ('process'|'compile', NULL?process) + `params` (JSON) columns via a v9 migration.

Ground yourself by reading the diff and touched code:

1. `git -C C:\Users\avidu\Projects\bhi-asyncnccompile diff` ? the full diff (backend/main.py, backend/infrastructure/database.py, backend/api/job_worker.py, + tests).
2. In `backend/main.py`: `\_CompileError`, `\_resolve\_compile`, the new async `compile\_highlights` endpoint, and `\_execute\_compile`. Compare to how `/api/process` (`process\_video`) + `\_execute\_processing` work.
3. In `backend/api/job\_worker.py`: `\_run\_claimed\_job` ? the new `is\_compile` dispatch branch.
4. In `backend/infrastructure/database.py`: `\_migrate\_to\_v9`, `create\_compile\_job`, and how `claim\_due\_job`/`get\_job` return the row (does the worker actually receive `kind`/`params`?).

Specifically probe and give a verdict on each:

- **Default-OFF / process unchanged**: a normal `/api/process` job has `kind` NULL. Does the worker still run it byte-identically (still `\_job\_to\_request`+`\_execute\_processing`, still cost+flywheel)? Trace `\_run\_claimed\_job`.
- **Migration safety**: is `\_migrate\_to\_v9` additive + idempotent (re-runnable ALTERS)? Is `SCHEMA\_VERSION` bumped to 9? Does a pre-v9 row (kind NULL) behave as 'process'? Any NOT NULL / default trap (note `jobs.video\_path` is NOT NULL ? what does a compile job store there)?
- **The worker receives params**: `claim\_due\_job` does `SELECT id, status` then returns `self.get\_job(id)` (SELECT *). Confirm `kind`+`params` reach `\_execute\_compile`. Does `get\_job` JSON-decode `params` or leave it a string ? and does `\_execute\_compile` handle whichever it is? (Check `get\_job`'s deserialization.)
- **Submit-vs-worker resolution**: `\_resolve\_compile` runs at submit (fast 4xx) AND in the worker. Is it safe to run twice? What if segments are deleted between submit and run ? does the worker fail cleanly (done+failed result) rather than crash? What if the stitch (ffmpeg) raises ? clean failure result, not a 500/worker-crash?
- **Error-shape parity**: the sync endpoint used to 500 on a stitch error; now it's a done job with `result.status=="failed"`. Is the result shape something the existing job-poll consumer can read (mirrors `\_execute\_processing`'s failed shape)?
- **Security**: `safe\_output\_filename` / path-traversal ? still enforced at submit AND worker? `owner\_id` (edge-auth tenant) ? captured on the compile job like on process? Any way an unauth/cross-tenant request slips through?
- **The breaking change**: 200?202 breaks any current sync caller (the deployed SPA). Is that correctly the intended fix (needs a paired SPA poll change), and are ALL in-repo callers/tests updated?

```
- get_job deserialization of params: does get_job try to json.loads only
result / player_pool, leaving params as a raw JSON string? _execute_compile does
json.loads(job.get("params") or "{}") ? confirm that's correct (i.e. params is NOT already
decoded, else double-decode/crash).
```

Return a concise findings list: severity (blocker/major/minor), exact file:line, why it's real, concrete fix. If nothing real, say so explicitly ? do not invent issues.

2. assistant (2026-06-22T17:42:13.933Z)

I'll start by pulling the worktree to ensure it's fresh, then read the diff and touched code.

3. user (2026-06-22T17:42:15.373Z)

On branch feat/async-compile

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

```
modified: backend/api/job_worker.py
modified: backend/infrastructure/database.py
modified: backend/main.py
modified: tests/test_api_endpoints.py
modified: tests/test_async_jobs.py
modified: tests/test_database.py
modified: tests/test_job_worker_loop.py
modified: tests/test_locale_analytics.py
```

no changes added to commit (use "git add" and/or "git commit -a")

---BRANCH---

feat/async-compile

---LOG---

```
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
5fb55e9 fix(cloudrun): install [storage-gcs] so the cloud worker can read/write gs:// (#337)
334ad0f feat(serving): M11 data-flywheel skeleton ? capture?shadow-eval?promote (default-OFF +
consent faked-deny) (#336)
6d3d054 feat(serving): SPA compute-picker ? per-request local/cloud override on /api/process
(item 3a) (#335)
2c8b3a3 feat(serving): P3a ? micromamba + conda-forge for the Cloud Run WASB env (no Anaconda
ToS) (#334)
```

4. assistant (2026-06-22T17:42:17.723Z)

Let me read the full diff and the key files in parallel.

5. user (2026-06-22T17:42:18.802Z)

<persisted-output>

Output too large (32.7KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\tool-results\br7ifhvm6.txt

Preview (first 2KB):

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
```

```
index ff8f537..5f4a5e4 100644
```

```
--- a/backend/api/job_worker.py
```

```
+++ b/backend/api/job_worker.py
```

```
@@ -85,15 +85,24 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
```

```
    import backend.main as _main
```

```
    db_instance = _main.db_instance
```

```
    job_id = job["id"]
```

```
- req = _main._job_to_request(job)
```

```
+ # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process' (NULL/default ?
```

```
the
```

```
+ # original /api/process pipeline). Both record their terminal state the same way; only
```

```

process
+   # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection usage/segments).
+   is_compile = job.get("kind") == "compile"
+   try:
-       result = _main._execute_processing(
-           req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
+   if is_compile:
+       result = _main._execute_compile(
+           job, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
+   else:
+       req = _main._job_to_request(job)
+       result = _main._execute_processing(
+           req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
+   if result.get("video_id"):
+       db_instance.set_job_video_id(job_id, result["video_id"])
+       db_instance.mark_job_done(job_id, result)
-       _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
-       _maybe_run_flywheel(job, result)    # M11 data-flywheel (default-OFF + consent-faked-
deny)
+   if not is_compile:
+       _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF; process jobs
only)
+       _maybe_run_flywheel(job, result)    # M11 data-flywheel (process jobs only)
+   except _main.JobWaiting as w:
+       # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
+       logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
...
</persisted-output>

```

6. assistant (2026-06-22T17:42:21.010Z)

Let me read the full diff from the saved file.

7. user (2026-06-22T17:42:22.176Z)

```

1      diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
2      index ff8f537..5f4a5e4 100644
3      --- a/backend/api/job_worker.py
4      +++ b/backend/api/job_worker.py
5      @@ -85,15 +85,24 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
6          import backend.main as _main
7          db_instance = _main.db_instance
8          job_id = job["id"]
9          req = _main._job_to_request(job)
10         + # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
11         + # original /api/process pipeline). Both record their terminal state the same way;
only process
12         + # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
13         + is_compile = job.get("kind") == "compile"
14         try:
15             result = _main._execute_processing(
16                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
17             + if is_compile:
18                 result = _main._execute_compile(
19                     job, progress=lambda stage: db_instance.set_job_progress(job_id,
stage))
20             + else:
21                 req = _main._job_to_request(job)
22                 result = _main._execute_processing(
23                     req, progress=lambda stage: db_instance.set_job_progress(job_id,
stage))
24                 if result.get("video_id"):
25                     db_instance.set_job_video_id(job_id, result["video_id"])

```

```

26         db_instance.mark_job_done(job_id, result)
27     -         _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
28     -         _maybe_run_flywheel(job, result)    # M11 data-flywheel (default-OFF +
consent-faked-deny)
29     +         if not is_compile:
30     +             _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF; process
jobs only)
31     +             _maybe_run_flywheel(job, result)    # M11 data-flywheel (process jobs
only)
32         except _main.JobWaiting as w:
33             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
34             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
35     diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
36     index d90b843..cc7f600 100644
37     --- a/backend/infrastructure/database.py
38     +++ b/backend/infrastructure/database.py
39     @@ -15,7 +15,7 @@ logger = logging.getLogger(__name__)
40     # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back
the DB up before
41     # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
42     # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
43     -SCHEMA_VERSION = 8
44     +SCHEMA_VERSION = 9
45
46
47     class SchemaTooNewError(RuntimeError):
48     @@ -140,7 +140,10 @@ class Database:
49
50         if version < 8:
51             self._migrate_to_v8(cursor)
52     -         # future: if version < 9: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 9
53     +
54     +         if version < 9:
55     +             self._migrate_to_v9(cursor)
56     +         # future: if version < 10: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 10
57
58         conn.commit()
59
60     @@ -414,6 +417,16 @@ class Database:
61         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
62         cursor.execute("PRAGMA user_version = 8")
63
64     +     def _migrate_to_v9(self, cursor):
65     +         # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the
original, NULL ?
66     +         # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
/api/compile
67     +         # is now 202+poll like /api/process so a slow ffmpeg stitch can't trip the edge
524). `params`
68     +         # is the kind-specific JSON payload a compile job carries
(segment_ids/output_filename/locale)
69     +         # since those don't map onto the process columns. Additive-only ALTERS;
idempotent.
70     +         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
71     +         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
72     +         cursor.execute("PRAGMA user_version = 9")
73     +
74         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
75             """Register or update a video row WITHOUT touching its child segments.
76
77     @@ -654,6 +667,26 @@ class Database:

```

```

78         f"Failed to create job {job_id}",
79     )
80
81     + def create_compile_job(self, job_id: str, video_id: str, video_path: str,
82     +                         segment_ids: List[int], output_filename: str,
83     +                         locale: Optional[str] = None, owner_id: Optional[str] =
None) -> bool:
84     +         """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
85     +         params (segment_ids / output_filename / locale) ride the JSON `params` column
since they don't
86     +         map onto the process columns; `video_path` is the target video's source ref
(the table's
87     +         NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
88     +         worker dispatches on `kind` and runs the ffmpeg stitch off the request
thread."""
89     +         params = json.dumps({
90     +             "segment_ids": list(segment_ids),
91     +             "output_filename": output_filename,
92     +             "locale": locale,
93     +         })
94     +         return self._execute_write(
95     +             "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale,
kind, params) "
96     +             "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
97     +             (job_id, video_path, video_id, owner_id, locale, params),
98     +             f"Failed to create compile job {job_id}",
99     +         )
100     +
101     def count_jobs_by_locale(self) -> Dict[str, int]:
102     """PMF analytics: how many jobs were submitted per UI locale. Unrecorded
locales (NULL ?
103     CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
104     diff --git a/backend/main.py b/backend/main.py
105     index f79bfd6..df08962 100644
106     --- a/backend/main.py
107     +++ b/backend/main.py
108     @@ -866,33 +866,38 @@ def query_play_segments(req: QueryRequest):
109     segments = db_instance.query_play_segments(req.video_id, filters)
110     return {"play_segments": segments}
111
112     -@app.post("/api/compile")
113     -def compile_highlights(req: CompileRequest):
114     -    video = db_instance.get_video(req.video_id)
115     +class _CompileError(Exception):
116     +    """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
117     +    maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
118     +    def __init__(self, status_code: int, detail: str):
119     +        super().__init__(detail)
120     +        self.status_code = status_code
121     +        self.detail = detail
122     +
123     +
124     +def _resolve_compile(video_id: str, segment_ids: List[int], output_filename: str):
125     +    """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
126     +    row could change between submit and run). Verifies the video exists, the requested
segments match
127     +    and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
128     +    so manual/legacy picks can't silently vanish), and the output name is traversal-

```

```

safe. Raises
129 + ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``. """
130 + video = db_instance.get_video(video_id)
131 + if not video:
132 -     raise HTTPException(status_code=404, detail="Indexed video record not found.")
133 -
134 - # Retrieve matching play segments from db
135 - all_segments = db_instance.query_play_segments(req.video_id, {})
136 - matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
137 -
138 - if not matched_segments:
139 -     raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
140 -
141 - # Reject path traversal / absolute output names (os.path.join would otherwise let
an
142 - # absolute or '..'-laden output_filename write anywhere on disk).
143 +     raise _CompileError(404, "Indexed video record not found.")
144 + # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
145     try:
146 -         out_name = safe_output_filename(req.output_filename)
147 +         out_name = safe_output_filename(output_filename)
148     except ValueError as e:
149 -         raise HTTPException(status_code=400, detail=str(e)) from e
150 +         raise _CompileError(400, str(e)) from e
151 +
152 + all_segments = db_instance.query_play_segments(video_id, {})
153 + matched = [s for s in all_segments if s["id"] in segment_ids]
154 + if not matched:
155 +     raise _CompileError(400, "No matching play segments found to stitch.")
156
157 - # The configured shot-count floor (stitching.min_shot_count) applies on the API
158 - # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
159 - # metadata are exempt: the floor filters known counts only, so explicitly
160 - # requested manual/legacy segments can't silently vanish under the default floor.
161     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
162     kept = []
163     for s in matched_segments:
164 +         for s in matched:
165             meta = s.get("metadata") or {}
166             sc = meta.get("shot_count") if isinstance(meta, dict) else None
167             try:
168 @@ -901,38 +906,91 @@ def compile_highlights(req: CompileRequest):
169                 except (TypeError, ValueError):
170                     pass # unparseable count -> treat as unknown, keep
171                     kept.append(s)
172 -             if len(kept) < len(matched_segments):
173 -                 logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
174 -                             f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
175 +             if len(kept) < len(matched):
176 +                 logger.info(f"[compile] stitching.min_shot_count=%s: dropped %d/%d segments
below the floor.",
177 +                             stitching.min_shot_count, len(matched) - len(kept), len(matched))
178             if not kept:
179 -                 raise HTTPException(status_code=400,
180 -                                     detail=f"All {len(matched_segments)} matching segments fall
below "
181 -                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
182 +                 raise _CompileError(400, f"All {len(matched)} matching segments fall below "

```

```

183      +
184      +     return video, kept, out_name
185      +
186      +
187      +@app.post("/api/compile", status_code=202)
188      +def compile_highlights(req: CompileRequest, request: Request):
189      +     """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).
190      +
191      +     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
192      +     can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
193      +     Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
194      +     got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
195      +     fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
196      +     {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
197      +     # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
198      +     try:
199      +         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
200      +     except edge_auth.EdgeAuthError as e:
201      +         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
202      +     try:
203      +         video, _kept, out_name = _resolve_compile(req.video_id, req.segment_ids,
req.output_filename)
204      +     except _CompileError as e:
205      +         raise HTTPException(status_code=e.status_code, detail=e.detail) from e
206      +
207      +     job_id = uuid.uuid4().hex
208      +     # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
209      +     # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
210      +     if not db_instance.create_compile_job(job_id, req.video_id, video["filepath"],
211      +         req.segment_ids, out_name, locale=req.locale,
212      +         owner_id=owner_id):
213      +         raise HTTPException(status_code=500, detail="Failed to persist compile job
record.")
214      +     _get_executor().submit(_drain_due_jobs)
215      +     return JSONResponse(
216      +         status_code=202,
217      +         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
218      +     )
219
220      -     # Extract start/end time pairs
221      -     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
222
223      -     # Run compiler with the configured stitching paddings + optional branding
watermark, localized to
224      -     # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
225      -     # with a script-appropriate font. Unknown/absent locale keeps the configured
default unchanged.
226      +def _execute_compile(job: Dict[str, Any], progress=None) -> Dict[str, Any]:
227      +     """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
228      +     and return the {status, filepath, download_url} result the SPA polls for. Returns a
229      +     ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)

```

```

230 + so the SPA always gets a clean verdict instead of a worker-crash error.""
231 + notify = progress or (lambda stage: None)
232 + params = json.loads(job.get("params") or "{}")
233 + video_id = job.get("video_id") or ""
234 + segment_ids = params.get("segment_ids") or []
235 + locale = params.get("locale")
236 +
237 + notify("resolving")
238 + try:
239 +     video, kept, out_name = _resolve_compile(video_id, segment_ids,
params.get("output_filename", ""))
240 +     except _CompileError as e:
241 +         return {"status": "failed", "message": e.detail, "video_id": video_id}
242 +
243 +     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
244 +     output_dir = getattr(getattr(global_config, "output", None), "output_dir",
"output") or "output"
245 -     localized_watermark = localize_watermark(stitching.watermark, req.locale)
246 +     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
247 +     # unknown/absent locale keeps the configured default unchanged.
248 +     localized_watermark = localize_watermark(stitching.watermark, locale)
249     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
250                             end_padding=stitching.end_padding,
watermark=localized_watermark)
251 +     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
252 +
253 +     notify("stitching")
254 +     try:
255 -         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
256 -         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
257 -         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
258 -         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
259 -         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
260 +         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
261 +         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
262 +         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
263         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
264         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
265 -         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
266 -         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
267 -         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
268 -         return {"status": "success", "filepath": out_path, "download_url":
download_url}
269 -     except Exception as e:
270 -         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
271 +     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
272 +         logger.error("[compile] stitching failed for video_id=%s: %s", video_id, e,
exc_info=True)
273 +         return {"status": "failed", "message": f"Highlight stitching failed: {e}",
"video_id": video_id}
274 +

```



```

275     + # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
276     + # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
277     + download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
278     + notify("finished")
279     + return {"status": "success", "filepath": out_path, "download_url": download_url,
"video_id": video_id}
280
281     # --- CLI Debug Utility & Orchestration ---
282     def run_cli_diagnose_clip(video_path: str, timestamp: float):
283     diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
284     index 4a5121c..1490d6a 100644
285     --- a/tests/test_api_endpoints.py
286     +++ b/tests/test_api_endpoints.py
287     @@ -309,10 +309,14 @@ def test_compile_success_with_mocked_compiler(env):
288         return f"output/{name}"
289
290         mp setattr(m, "VideoCompiler", _FakeCompiler)
291     + # Async (#329): submit ? 202 + job_id; the inline executor runs the stitch; poll
for the result.
292         r = client.post("/api/compile", json={
293             "video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"})
294     - assert r.status_code == 200
295     - body = r.json()
296     + assert r.status_code == 202
297     + job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
298     + assert job["status"] == "done"
299     + body = job["result"]
300     + assert body["status"] == "success"
301     + assert body["filepath"] == "output/out.mp4"
302     + # the browser-reachable URL is returned so the SPA needn't hand-build it (P6)
303     + assert body["download_url"] == "/api/highlights/vid/out.mp4"
304     @@ -347,13 +351,79 @@ def
test_compile_resolves_bucket_source_to_local_before_stitching(env):
305         mp setattr(m, "VideoCompiler", _CaptureCompiler)
306         r = client.post("/api/compile", json={
307             "video_id": "bvid", "segment_ids": [sid], "output_filename": "reel.mp4"})
308     - assert r.status_code == 200
309     + assert r.status_code == 202
310     + job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
311     + assert job["status"] == "done" and job["result"]["status"] == "success"
312     + # The stitcher received a FETCHED LOCAL path, NOT the gs:// URI.
313     + assert _CaptureCompiler.last_raw is not None
314     + assert not _CaptureCompiler.last_raw.startswith("gs://")
315     + assert os.path.basename(_CaptureCompiler.last_raw) == "x.mp4"
316
317
318     +def _fake_ok_compiler():
319     +     class _FakeCompiler:
320     +     def __init__(self, *a, **k):
321     +         pass
322     +
323     +     def compile_highlights(self, raw, pairs, name):
324     +         return f"output/{name}"
325     +     return _FakeCompiler
326
327
328     +def test_compile_returns_202_with_poll_envelope(env):
329     +     """#329: /api/compile is async ? submit returns 202 {job_id, status:'queued', poll}
like /api/process."""
330     +     client, db, tmp_path, mp = env
331     +     db.add_video("vid", str(_video(tmp_path)), "processed", [])
332     +     sid = db.add_play_segment("vid", 1.0, 4.0)
333     +     mp setattr(m, "VideoCompiler", _fake_ok_compiler())
334     +     r = client.post("/api/compile", json={

```

```

335 +         "video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"})
336 +     assert r.status_code == 202
337 +     body = r.json()
338 +     assert body["status"] == "queued"
339 +     assert body["poll"] == f"/api/jobs/{body['job_id']}"
340 +
341 +
342 +def test_compile_unknown_video_404_at_submit(env):
343 +     """The cheap checks still fast-fail at SUBMIT (4xx), before any job is queued."""
344 +     client, db, tmp_path, mp = env
345 +     r = client.post("/api/compile", json={
346 +         "video_id": "nope", "segment_ids": [1], "output_filename": "out.mp4"})
347 +     assert r.status_code == 404
348 +
349 +
350 +def test_compile_no_matching_segments_400_at_submit(env):
351 +     client, db, tmp_path, mp = env
352 +     db.add_video("vid", str(_video(tmp_path)), "processed", [])
353 +     r = client.post("/api/compile", json={
354 +         "video_id": "vid", "segment_ids": [999], "output_filename": "out.mp4"})
355 +     assert r.status_code == 400
356 +
357 +
358 +def test_compile_stitch_failure_marks_job_done_with_failed_result(env):
359 +     """A stitch/ffmpeg error becomes a clean done+failed RESULT (not a worker crash /
360 +     'failed' status),
361 +     so the SPA always gets a verdict + message rather than a silent hang or an opaque
362 +     error."""
363 +     client, db, tmp_path, mp = env
364 +     db.add_video("vid", str(_video(tmp_path)), "processed", [])
365 +     sid = db.add_play_segment("vid", 1.0, 4.0)
366 +
367 +     class _BoomCompiler:
368 +         def __init__(self, *a, **k):
369 +             pass
370 +
371 +         def compile_highlights(self, raw, pairs, name):
372 +             raise RuntimeError("ffmpeg exploded")
373 +
374 +     mp.setattr(m, "VideoCompiler", _BoomCompiler)
375 +     r = client.post("/api/compile", json={
376 +         "video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"})
377 +     assert r.status_code == 202
378 +     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
379 +     assert job["status"] == "done" # the WORKER finished cleanly
380 +     assert job["result"]["status"] == "failed" # the pipeline VERDICT is a failure
381 +     assert "ffmpeg exploded" in job["result"]["message"]
382 +
383 +
384 +def test_health_ok(env):
385 +     client = env[0]
386 +     r = client.get("/api/health")
387 +
388 +@@ -387,10 +457,13 @@ def test_compile_enforces_config_min_shot_count(env):
389 +     return f"output/{name}"
390 +
391 +
392 +     mp.setattr(m, "VideoCompiler", _FakeCompiler)
393 +     # Async (#329): the inline executor runs the stitch during the POST, so the floor
394 +     still applies.
395 +     r = client.post("/api/compile", json={
396 +         "video_id": "vid", "segment_ids": [s_low, s_high, s_unknown],
397 +         "output_filename": "reel.mp4"})
398 +     assert r.status_code == 200
399 +     assert r.status_code == 202
400 +     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
401 +     assert job["status"] == "done" and job["result"]["status"] == "success"

```

```

397         assert (10.0, 14.0) in pairs_seen and (20.0, 24.0) in pairs_seen
398         assert (1.0, 4.0) not in pairs_seen
399
400     @@ -432,9 +505,13 @@ def test_compile_uses_configured_stitching_paddings(env):
401         return MagicMock(returncode=0, stdout=b"", stderr=b"")
402
403         with patch.object(subprocess, "run", side_effect=fake_run):
404             + # The inline executor runs the stitch (under this patch) during the POST, so
the ffmpeg
405             + # trim commands are captured here; then the job is already 'done'.
406             r = client.post("/api/compile", json={
407                 "video_id": "vid", "segment_ids": [sid], "output_filename": "reel.mp4"})
408             - assert r.status_code == 200
409             + job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
410             + assert r.status_code == 202
411             + assert job["status"] == "done" and job["result"]["status"] == "success"
412
413             trim = next(c for c in calls if c[0] == "ffmpeg" and "-ss" in c)
414             # 10.0 - begin_padding 2.0 and 14.0 + end_padding 3.0; the unwired constructor
415 diff --git a/tests/test_async_jobs.py b/tests/test_async_jobs.py
416 index 0ec15f7..c6ffd97 100644
417 --- a/tests/test_async_jobs.py
418 +++ b/tests/test_async_jobs.py
419 @@ -9,6 +9,7 @@ Contract under test:
420 - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
421 - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in
place
422     """
423     +import json
424     import sqlite3
425     import time
426     from concurrent.futures import ThreadPoolExecutor
427     @@ -79,13 +80,13 @@ class _RaisingSeg:
428     def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
429         db = Database(str(tmp_path / "fresh.db"))
430         with db._get_connection() as conn:
431             - # v8 = job compute-picker; v7 = job UI locale (PMF analytics); v6 = cost ledger
(M8);
432             + # v9 = async compile (kind/params); v8 = compute-picker; v7 = UI locale; v6 =
cost ledger;
433             # jobs at v2/v3, user_models v5.
434             - assert conn.execute("PRAGMA user_version").fetchone()[0] == 8
435             + assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
436             cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
437             assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <=
cols
438             assert {"policy", "next_poll_at"} <= cols # v3 self-scheduling columns
439             - assert {"locale", "compute"} <= cols # v7 locale, v8 compute-picker
choice
440             + assert {"locale", "compute", "kind", "params"} <= cols # v7 locale, v8 compute,
v9 compile job
441
442
443     def test_migration_upgrades_v1_db_in_place(tmp_path):
444     @@ -104,9 +105,9 @@ def test_migration_upgrades_v1_db_in_place(tmp_path):
445         conn.commit()
446         conn.close()
447
448         - db = Database(path) # ladder should take it 1 ? 8
449         + db = Database(path) # ladder should take it 1 ? 9
450         with db._get_connection() as c:
451             - assert c.execute("PRAGMA user_version").fetchone()[0] == 8
452             + assert c.execute("PRAGMA user_version").fetchone()[0] == 9
453             assert c.execute(
454                 "SELECT name FROM sqlite_master WHERE type='table' AND

```

```

name='jobs')).fetchone()
455         # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
456     @@ -311,6 +312,43 @@ def test_get_unknown_job_is_404(env):
457         assert client.get("/api/jobs/deadbeef").status_code == 404
458
459
460     +def test_create_compile_job_persists_kind_and_params(tmp_path):
461     +    """#329: an async compile job rides the jobs table with kind='compile' + a JSON
params payload
462     +    (segment_ids/output_filename/locale); a process job leaves kind NULL (? process ?
byte-identical)."""
463     +    db = Database(str(tmp_path / "jobs.db"))
464     +    assert db.create_compile_job("cj", "vidX", "gs://b/x.mp4", [1, 2, 3], "reel.mp4",
locale="kn")
465     +    job = db.get_job("cj")
466     +    assert job["kind"] == "compile"
467     +    assert job["video_id"] == "vidX"
468     +    params = json.loads(job["params"])
469     +    assert params["segment_ids"] == [1, 2, 3]
470     +    assert params["output_filename"] == "reel.mp4"
471     +    assert params["locale"] == "kn"
472     +    db.create_job("pj", "/v.mp4") # a process job
473     +    assert db.get_job("pj")["kind"] is None
474     +
475     +
476     +def test_worker_dispatches_compile_kind_to_execute_compile(env):
477     +    """The claimed-job dispatcher routes kind='compile' to _execute_compile (NOT
_execute_processing),
478     +    records the result as 'done', and skips the process-only cost/flywheel hooks."""
479     +    _client, db, _tmp_path, mp = env
480     +    seen = {}
481     +
482     +    def _fake_compile(job, progress=None):
483     +        seen["id"] = job["id"]
484     +        return {"status": "success", "video_id": "vidX", "download_url":
"/api/highlights/vidX/r.mp4"}
485     +
486     +    mp.setattr(m, "_execute_compile", _fake_compile)
487     +    mp.setattr(m, "_execute_processing",
lambda *a, **k: pytest.fail("a compile job must not hit
_execute_processing"))
488     +
489     +    db.create_compile_job("cj2", "vidX", "/v.mp4", [1], "r.mp4")
490     +    m._run_claimed_job(db.get_job("cj2"))
491     +    assert seen["id"] == "cj2"
492     +    done = db.get_job("cj2")
493     +    assert done["status"] == "done"
494     +    assert done["result"]["download_url"] == "/api/highlights/vidX/r.mp4"
495     +
496     +
497     def test_create_job_persists_and_reconstructs_compute(tmp_path):
498     +    """v8: the compute-picker choice must round-trip create_job ? get_job ?
_job_to_request so the
499     +    worker honours it across the async submit?reconstruct boundary. NULL ? server
default (the
500     +    diff --git a/tests/test_database.py b/tests/test_database.py
501     +    index 4bfc127..89cbadb 100644
502     +    --- a/tests/test_database.py
503     +    +++ b/tests/test_database.py
504     +    @@ -85,9 +85,9 @@ def test_schema_version_is_current(tmp_path):
505     +        conn = sqlite3.connect(str(db_path))
506     +        version = conn.execute("PRAGMA user_version").fetchone()[0]
507     +        conn.close()
508     +    -    # v8 = job compute-picker choice; v7 = job UI locale (PMF analytics); v6 = cost
ledger (M8).

```

```

509     + # v9 = async compile job (kind/params); v8 = compute-picker; v7 = job UI locale; v6
= cost ledger.
510     # Bump with each migration.
511     - assert version == 8
512     + assert version == 9
513
514
515     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
516     @@ -112,7 +112,7 @@ def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
517         assert "jobs" in tables # v2 (async job model, #16)
518         assert "user_models" in tables # v5 (per-user model store)
519         assert "pricebook" in tables # v6 (per-job cost ledger, M8)
520     - assert version == 8 # v8 = job compute-picker; v7 = job UI locale; v6 = cost
ledger (M8)
521     + assert version == 9 # v9 = async compile (kind/params); v8 = compute-picker;
v7 = locale
522
523
524     def test_candidate_round_trip(tmp_path):
525     @@ -212,7 +212,7 @@ def test_v4_existing_writers_leave_user_id_null(tmp_path):
526
527
528     def test_migration_v3_db_upgrades_to_head_preserving_rows(tmp_path):
529     - """A real v3 DB (jobs + candidates already populated) upgrades cleanly to the head
(v8): the new
530     + """A real v3 DB (jobs + candidates already populated) upgrades cleanly to the head
(v9): the new
531         columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERS."""
532         db_path = tmp_path / "legacy_v3.db"
533         db = Database(str(db_path)) # fresh DB is born at the current version; simulate a
v3 state
534     @@ -232,10 +232,10 @@ def
test_migration_v3_db_upgrades_to_head_preserving_rows(tmp_path):
535         conn.close()
536         assert "user_id" not in _table_cols(db_path, "videos")
537
538     - # Re-open: the migration ladder must carry the v3 DB up to the head (v8) without
data loss.
539     + # Re-open: the migration ladder must carry the v3 DB up to the head (v9) without
data loss.
540         db2 = Database(str(db_path))
541         conn = sqlite3.connect(str(db_path))
542     - assert conn.execute("PRAGMA user_version").fetchone()[0] == 8
543     + assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
544         tables = {r[0] for r in conn.execute(
545             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
546         assert "user_models" in tables # v5
547     @@ -269,12 +269,12 @@ def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
548         conn.close()
549
550         # Re-opening re-runs v3 (jobs.policy/next_poll_at), v4
(videos/candidate_segments.user_id),
551     - # v5 (user_models), v6 (jobs cost columns + pricebook), v7 (jobs.locale) and v8
(jobs.compute)
552     - # over artifacts that ALREADY exist ? must NOT raise (re-runnable ALTERS / INSERT
OR IGNORE),
553     - # and must reach v8.
554     + # v5 (user_models), v6 (jobs cost columns + pricebook), v7 (jobs.locale), v8
(jobs.compute) and
555     + # v9 (jobs.kind/params) over artifacts that ALREADY exist ? must NOT raise (re-
runnable ALTERS /
556     + # INSERT OR IGNORE), and must reach v9.
557         Database(str(db_path))
558         conn = sqlite3.connect(str(db_path))

```

```

559 - assert conn.execute("PRAGMA user_version").fetchone()[0] == 8
560 + assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
561     conn.close()
562     # Columns/tables are all present after the clean re-apply.
563     assert "policy" in _table_cols(db_path, "jobs")
564 diff --git a/tests/test_job_worker_loop.py b/tests/test_job_worker_loop.py
565 index fbc29d1..017709a 100644
566 --- a/tests/test_job_worker_loop.py
567 +++ b/tests/test_job_worker_loop.py
568 @@ -268,16 +268,16 @@ def test_waiting_columns_present(tmp_path):
569     assert db.get_job("j")["status"] == "waiting"
570
571
572 -def test_fresh_db_is_v8(tmp_path):
573 +def test_fresh_db_is_v9(tmp_path):
574     db = Database(str(tmp_path / "v.db"))
575     with db._get_connection() as conn:
576         # Head of the migration ladder (bump when database.py adds a new `if version <
577         N`).
578 - assert conn.execute("PRAGMA user_version").fetchone()[0] == 8
579 + assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
580
581     def test_legacy_db_upgrades_to_head(tmp_path):
582         # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL
583 ordered
584 - # migration ladder up to the current head (v8) without error ? this exercises the
585 v0?v8
586 + # migration ladder up to the current head (v9) without error ? this exercises the
587 v0?v9
588     # path including the v1 `candidate_segments` CREATE that the later v4 ALTER depends
589 on.
590     path = str(tmp_path / "legacy.db")
591     conn = sqlite3.connect(path)
592 @@ -289,4 +289,4 @@ def test_legacy_db_upgrades_to_head(tmp_path):
593     conn.close()
594     db = Database(path)
595     with db._get_connection() as c:
596 - assert c.execute("PRAGMA user_version").fetchone()[0] == 8
597 + assert c.execute("PRAGMA user_version").fetchone()[0] == 9
598 diff --git a/tests/test_locale_analytics.py b/tests/test_locale_analytics.py
599 index 8dab0e9..eba12eb 100644
600 --- a/tests/test_locale_analytics.py
601 +++ b/tests/test_locale_analytics.py
602 @@ -30,7 +30,7 @@ def _user_version(path):
603     def test_fresh_db_v7_has_jobs_locale_column(tmp_path):
604         path = str(tmp_path / "fresh.db")
605         Database(path)
606 - assert _user_version(path) == 8 # head: v7 added locale, v8 added compute
607 + assert _user_version(path) == 9 # head: v7 added locale, v8 compute, v9 compile
608 job
609     assert "locale" in _cols(path, "jobs")
610
611
612 @@ -46,8 +46,8 @@ def test_v6_db_upgrades_to_v7_and_backfills_locale_null(tmp_path):
613     conn.close()
614     assert "locale" not in _cols(path, "jobs")
615
616 - Database(path) # re-open ? the ladder runs _migrate_to_v7 (+ v8) up to the head
617 - assert _user_version(path) == 8
618 + Database(path) # re-open ? the ladder runs _migrate_to_v7 (+ v8, v9) up to the
619 head
620 + assert _user_version(path) == 9
621     assert "locale" in _cols(path, "jobs")
622     # The prior row survived the ALTER and back-filled to NULL (not dropped, not

```

```
defaulted to a value).
617         conn = sqlite3.connect(path)
618
```

8. assistant (2026-06-22T17:42:25.082Z)

I have the full diff. Now let me examine the surrounding code in `main.py`, `job\_worker.py`, and `database.py` to verify the details ? especially `get\_job` deserialization, `\_job\_to\_request`, `claim\_due\_job`, `edge\_auth`, and how `/api/process` mirrors this.

9. user (2026-06-22T17:42:26.023Z)

```
600         if filters.get("duration_min"):
601             query += " AND r.duration >= ?"
602             params.append(filters["duration_min"])
603
604         if filters.get("server"):
605             query += " AND r.server = ?"
606             params.append(filters["server"])
607
608         if filters.get("receiver"):
609             query += " AND r.receiver = ?"
610             params.append(filters["receiver"])
611
612         if filters.get("event_type"):
613             # Check if there is an event of a specific type inside this segment
614             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
615             params.append(filters["event_type"])
616
617         segments_list = []
618         with self._connect() as conn:
619             cursor = conn.cursor()
620             rows = cursor.execute(query, params).fetchall()
621
622             for row in rows:
623                 r_dict = dict(row)
624                 r_id = r_dict["id"]
625                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
626
627                 # Fetch associated events
628                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
629                 r_dict["events"] = []
630                 for er in event_rows:
631                     e_dict = dict(er)
632                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
633                     r_dict["events"].append(e_dict)
634
635                 segments_list.append(r_dict)
636
637         return segments_list
638
639     def clear_video_data(self, video_id: str):
640         with self._connect() as conn:
641             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
642             conn.commit()
643
644     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
645     # Writers follow the repo convention (swallow + logger.error, return bool); the
646     # worker logs loudly on a failed transition so a job can't silently wedge.
647
648     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
649                   player_pool: Optional[List[str]] = None,
650                   max_frames: Optional[int] = None,
```

```

651         policy: Optional[Dict[str, Any]] = None,
652         owner_id: Optional[str] = None,
653         locale: Optional[str] = None,
654         compute: Optional[str] = None) -> bool:
655     """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
656     (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
657     (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
658     ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
659     ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
660     honours; NULL ? the server default (byte-identical to pre-picker behaviour)."""
661     return self._execute_write(
662         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
663         "owner_id, locale, compute) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?)",
664         (job_id, video_path, segmenter,
665         json.dumps(player_pool) if player_pool is not None else None,
666         max_frames, json.dumps(policy) if policy is not None else None, owner_id,
667         locale, compute),
668         f"Failed to create job {job_id}",
669     )
670     def create_compile_job(self, job_id: str, video_id: str, video_path: str,
671         segment_ids: List[int], output_filename: str,
672         locale: Optional[str] = None, owner_id: Optional[str] = None)
-> bool:
673     """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
674     params (segment_ids / output_filename / locale) ride the JSON `params` column
since they don't
675     map onto the process columns; `video_path` is the target video's source ref (the
table's
676     NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
677     worker dispatches on `kind` and runs the ffmpeg stitch off the request
thread."""
678     params = json.dumps({
679         "segment_ids": list(segment_ids),
680         "output_filename": output_filename,
681         "locale": locale,
682     })
683     return self._execute_write(
684         "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
685         "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
686         (job_id, video_path, video_id, owner_id, locale, params),
687         f"Failed to create compile job {job_id}",
688     )
689
690     def count_jobs_by_locale(self) -> Dict[str, int]:
691     """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
692     CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
693     with self._connect() as conn:
694         rows = conn.cursor().execute(
695             "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
696             "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
697         ).fetchall()
698     return {str(r[0]): int(r[1]) for r in rows}
699

```



```

700     def mark_job_running(self, job_id: str) -> bool:
701         return self._execute_write(
702             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
703             "progress='starting' WHERE id = ?", (job_id,),
704             f"Failed to mark job {job_id} running",
705         )
706
707     def set_job_progress(self, job_id: str, progress: str) -> bool:
708         return self._execute_write(
709             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
710             f"Failed to set progress for job {job_id}",
711         )
712
713     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
714         return self._execute_write(
715             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
716             f"Failed to set video_id for job {job_id}",
717         )
718
719     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
720         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
721         validation) is inside `result` ? job status 'done' means 'the worker
722         finished'."""
723         return self._execute_write(
724             "UPDATE jobs SET status='done', progress='finished', result=?, "
725             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
726             (json.dumps(result), job_id),
727             f"Failed to mark job {job_id} done",
728         )
729
730     def mark_job_failed(self, job_id: str, error: str) -> bool:

```

10. assistant (2026-06-22T17:42:28.909Z)

Now let me look at `get\_job` and `claim\_due\_job` to confirm what fields are deserialized and returned.

11. user (2026-06-22T17:42:29.872Z)

```

729     def mark_job_failed(self, job_id: str, error: str) -> bool:
730         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
731         return self._execute_write(
732             "UPDATE jobs SET status='failed', error=?, "
733             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
734             f"Failed to mark job {job_id} failed",
735         )
736
737     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
738         """Fetch one job; `result`/`player_pool` JSON deserialized."""
739         with self._connect() as conn:
740             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
741             (job_id,)).fetchone()
742             if not row:
743                 return None
744             d = dict(row)
745             d["result"] = json.loads(d["result"]) if d["result"] else None
746             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
747             None
748             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
749             return d
750
751     # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
752     -----
753     def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) ->
754     Dict[str, float]:

```

```

751         """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
752 GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''`.
753 An unknown version ? ``{}`` (so every cost computes to 0.0)."""
754 gt = gpu_type or ""
755 rates: Dict[str, float] = {}
756 with self._connect() as conn:
757     rows = conn.cursor().execute(
758         "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version
= ?",
759         (version,)).fetchall()
760     for r in rows:
761         res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
762         if res == billing.GPU_SECONDS:
763             if row_gt == gt: # the GPU rate for THIS gpu_type
764                 rates[res] = price
765             else:
766                 rates[res] = price # wall/egress: gpu_type-independent
767     return rates
768
769     def record_job_cost(self, job_id: str, *, usage: Dict[str, float],
pricebook_version: str,
770                         gpu_type: Optional[str] = None, compute_backend: Optional[str] =
None,
771                         owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
772         """Compute the job's cost from ``usage`` x the pricebook + persist every cost
column.
773         Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
774         ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
775         rates = self.get_pricebook_rates(pricebook_version, gpu_type)
776         # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
777         # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
778         # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
779         gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
780         unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
781         if unpriced_gpu:
782             logger.warning(
783                 "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
784                 "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
785                 gpu_type, pricebook_version, gpu_amt)
786         cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
787         if unpriced_gpu:
788             breakdown["unpriced_gpu_seconds"] = gpu_amt
789         if margin:
790             marked = billing.apply_margin(cost_usd, margin)
791             breakdown["margin"] = round(marked - cost_usd, 6) # keep ?(breakdown) ==
cost_usd
792             cost_usd = marked
793         bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
794         ok = self._execute_write(
795             "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
796             "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
797             "cost_breakdown_json=? WHERE id = ?",

```

```

798         (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
799         usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
800         pricebook_version, cost_usd, json.dumps(breakdown), job_id),
801         f"Failed to record cost for job {job_id}")
802     if not ok:
803         return None
804     return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
805           "pricebook_version": pricebook_version}
806
807     def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
808         """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
809         ``None`` until
810         recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
811         with self._connect() as conn:
812             row = conn.cursor().execute(
813                 "SELECT id, video_id, owner_id, compute_backend, gpu_type,
814                 gpu_active_seconds, "
815                 "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd,
816                 "
817                 "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
818             if not row:
819                 return None
820             d = dict(row)
821             raw = d.pop("cost_breakdown_json", None)
822             d["cost_breakdown"] = json.loads(raw) if raw else {}
823             return d
824
825     def get_video_cost(self, video_id: str) -> Dict[str, Any]:
826         """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
827         A video is
828         1:1 with an infer job, but a re-ingest can produce several ? sum them."""
829         with self._connect() as conn:
830             rows = conn.cursor().execute(
831                 "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
832                 "WHERE video_id = ?", (video_id,)).fetchall()
833             jobs: List[Dict[str, Any]] = []
834             total = 0.0
835             for r in rows:
836                 cost = r["cost_usd"]
837                 if cost is not None:
838                     total += float(cost)
839                 jobs.append({
840                     "job_id": r["id"], "cost_usd": cost, "pricebook_version":
841                     r["pricebook_version"],
842                     "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
843                     r["cost_breakdown_json"] else {},
844                 })
845             return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
846
847     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
848     -----
849     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
850         """Attach/replace a job's JSON ExecutionPolicy."""
851         return self._execute_write(
852             "UPDATE jobs SET policy=? WHERE id = ?",
853             (json.dumps(policy) if policy is not None else None, job_id),
854             f"Failed to set policy for job {job_id}",
855         )
856
857     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
858     None) -> bool:
859         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
860         releasing the
861         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
862         ``next_poll_at`` is

```

```

853         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP.""
854         return self._execute_write(
855             "UPDATE jobs SET status='waiting', "
856             "next_poll_at=datetime('now', ?), "
857             "progress=COALESCE(?, progress) WHERE id = ?",
858             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
859             f"Failed to set job {job_id} waiting",
860         )
861
862     def seconds_until_next_due(self) -> Optional[float]:
863         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
864         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
865         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
866         try:
867             with self._connect() as conn:
868                 row = conn.cursor().execute(
869                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
870                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
871                     "FROM jobs WHERE status='waiting'").fetchone()
872                 if row is None or row["secs"] is None:
873                     return None
874                 return max(0.0, float(row["secs"]))
875         except Exception as e:
876             logger.error(f"Failed to compute next-due delay: {e}")
877             return None
878
879     def claim_due_job(self) -> Optional[Dict[str, Any]]:
880         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
881         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
882         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
883         first. Returns the claimed job dict, or None if nothing is runnable."""
884         try:
885             with self._connect() as conn:
886                 cur = conn.cursor()
887                 row = cur.execute(
888                     "SELECT id, status FROM jobs WHERE status='queued' "

```

12. assistant (2026-06-22T17:42:32.916Z)

Confirmed: `get\_job` deserializes `result`, `player\_pool`, `policy` ? but NOT `params`. So `params` stays a raw JSON string, and `\_execute\_compile` correctly does `json.loads(job.get("params") or "{}")`. That's consistent. Let me see the rest of `claim\_due\_job`.

13. user (2026-06-22T17:42:34.092Z)

```

889         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
890         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
891         if not row:
892             return None
893         cur.execute(
894             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
895             "WHERE id=? AND status=?", (row["id"], row["status"]))
896         conn.commit()
897         if cur.rowcount != 1:

```

```

898             return None                                     # lost the race to a concurrent
worker
899         return self.get_job(row["id"])
900     except Exception as e:
901         logger.error(f"Failed to claim a due job: {e}")
902         return None
903
904     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
905         """Crash recovery: any job still queued/running from a previous process is dead
906         (the executor is in-process). Mark failed so clients see a terminal state, not a
907         hang. Called once at startup. Returns the number of jobs swept."""
908         try:
909             with self._connect() as conn:
910                 cur = conn.cursor()
911                 cur.execute(
912                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
913                     "WHERE status IN ('queued', 'running')", (reason,))
914                 conn.commit()
915                 return cur.rowcount
916         except Exception as e:
917             logger.error(f"Failed to sweep stale jobs: {e}")
918             return 0
919
920         # --- Per-user model store (PERSONALIZATION_PLAN.md $2, v5) -----
921         # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
922         # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
923         # serving.py) resolves them, and wiring that bridge into the production decision
seam
924         # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
925         # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
926         # JSON-serialised dicts; the rest are scalar columns.
927
928     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,

```

14. assistant (2026-06-22T17:42:41.839Z)

`claim\_due\_job` returns `self.get\_job(row["id"])`, which is `SELECT \*` ? so `kind` and `params` both reach the worker. Good. Now let me read the relevant portions of `main.py` ? the `process\_video` endpoint (to compare owner_id capture + 202 envelope), `\_job\_to\_request`, `\_execute\_processing`'s failed result shape, and the imports (`Request`, `JSONResponse`, `edge\_auth`, `uuid`).

15. user (2026-06-22T17:42:43.901Z)

```

1:import os
2:import sys
3:import argparse
4:import csv
5:import json
6:import logging
7:import math
8:import shutil
9:import socket
10:import tempfile
11:import threading
12:import uuid
13:import webbrowser
14:from importlib.metadata import PackageNotFoundError, version as _pkg_version
15:from urllib.parse import quote
16:import uvicorn
17:from dotenv import load_dotenv
23:from backend.utils.app_home import env_file_to_load, resolve_output_dir
34:from fastapi import FastAPI, HTTPException, Request
35:from fastapi.middleware.cors import CORSMiddleware

```

```

36:from starlette.middleware.trustedhost import TrustedHostMiddleware
37:from pydantic import BaseModel
38:from typing import List, Dict, Any, Optional, Tuple
40:from fastapi.staticfiles import StaticFiles
41:from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
42:from backend.pipeline.validators.base import registry
43:from backend.infrastructure.database import Database
44:from backend.pipeline.segmenters.base import segmenter_registry
45:from backend.pipeline.stitcher.compiler import VideoCompiler
46:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
47:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
48:from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
49:from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
50:from backend.utils.single_instance import acquire_lock # P0e: one instance per database
51:from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
52:from backend.config import ( # new typed config
60:from backend.results import Failure # standardized error/result contract
61:from backend.reporting import emit_indexer_report
62:from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
63:from backend import billing # M8: per-job cost ledger (USD internal / INR display)
64:from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)
243:from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
282:class CompileRequest(BaseModel):
594:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
724:def process_video(req: ProcessRequest, request: Request):
736:    owner_id = edge_auth.resolve_tenant(request.headers, global_config)
769:    return JSONResponse(
827:from backend.api import uploads as uploads_api
930:    owner_id = edge_auth.resolve_tenant(request.headers, global_config)
946:    return JSONResponse(

```

16. assistant (2026-06-22T17:42:46.642Z)

Now let me read `process\_video`, `\_job\_to\_request`, `\_execute\_processing`'s failed shape, and the `CompileRequest` model.

17. user (2026-06-22T17:42:47.727Z)

```

282    class CompileRequest(BaseModel):
283        video_id: str
284        segment_ids: List[int]
285        output_filename: str
286        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
287        # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
288        locale: Optional[str] = None
289
290    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
291        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
292
293        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
294        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
295        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
296        """
297        if segments:
298            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
299        else:
300            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
301

```

18. user (2026-06-22T17:42:50.701Z)

```
700         _finalize_reingest(video_id, segments, play_wm, cand_wm)
701
702         response = {
703             "video_id": video_id,
704             "status": "success",
705             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
706             "results": [r.model_dump() for r in val_results],
707             "play_segments": segments,
708         }
709         return response
710     finally:
711         # Always emit the per-video observability report (next to the video, or to
712         # report.output_dir). Never raises. Surface the paths in the response.
713         paths = emit_indexer_report(
714             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
715             val_results=val_results, val_context=val_context,
716             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
717             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
718         )
719         if paths and isinstance(response, dict):
720             response["reports"] = paths
721
722
723     @app.post("/api/process", status_code=202)
724     def process_video(req: ProcessRequest, request: Request):
725         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
726
727         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
728         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
729         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
730         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
731         """
732         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
733         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
734         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
735         try:
736             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
737         except edge_auth.EdgeAuthError as e:
738             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
739
740         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
741         try:
742             validate_input_path(req.video_path, allowed_root=allowed_root)
743             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
744             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
745             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
746         except ValueError as e:
747             raise HTTPException(status_code=400, detail=str(e)) from e
748         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
749         # (input_ref.key ? e.g. local:/// stripped to the bare path), not the raw URI
string. A
750         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
751         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
```

```

runs for every
752     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
753     # rejects a non-local URI as out-of-root.
754     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
755         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
756     if req.segmenter and not segmenter_registry.get(req.segmenter):
757         available = list(segmenter_registry.list_available().keys())
758         raise HTTPException(
759             status_code=400,
760             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
761         )
762
763     job_id = uuid.uuid4().hex
764     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
765                                 req.player_pool, req.max_frames, owner_id=owner_id,
766                                 locale=req.locale, compute=req.compute):
767         raise HTTPException(status_code=500, detail="Failed to persist job record.")
768     _get_executor().submit(_drain_due_jobs)
769     return JSONResponse(
770         status_code=202,
771         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
772     )
773
774
775     @app.get("/api/jobs/{job_id}/cost")
776     def get_job_cost(job_id: str):
777         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
778         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
779         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
780         cost = db_instance.get_job_cost(job_id)
781         if cost is None:
782             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
783         fx = global_config.billing.fx_inr_per_usd
784         usd = cost.get("cost_usd")
785         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
786         cost["fx_inr_per_usd"] = fx
787         return cost
788
789
790     @app.get("/api/videos/{video_id}/cost")
791     def get_video_cost(video_id: str):
792         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
793         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
794         agg = db_instance.get_video_cost(video_id)
795         fx = global_config.billing.fx_inr_per_usd
796         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
797         agg["fx_inr_per_usd"] = fx
798         return agg
799
800
801     @app.get("/api/jobs/{job_id}")
802     def get_job(job_id: str):
803         """Job state: {status, progress, result, reports}. `status` = execution state
804         (queued|running|done|failed); the pipeline verdict is result["status"]."""
805         job = db_instance.get_job(job_id)
806         if not job:
807             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
808         result = job.get("result")

```



```

809         return {
810             "job_id": job["id"],
811             "status": job["status"],
812             "progress": job.get("progress"),
813             "video_id": job.get("video_id"),
814             "error": job.get("error"),
815             "result": result,
816             "reports": (result or {}).get("reports"),
817             "created_at": job.get("created_at"),
818             "started_at": job.get("started_at"),
819             "finished_at": job.get("finished_at"),
820         }
821
822     # --- Chunked upload (B2, PR-2) -----
823     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
824     # in
825     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
826     # via
827     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
828     # sequential-part logic, and abort cleanup ? is unchanged.
829     from backend.api import uploads as uploads_api
830
831     app.include_router(uploads_api.router)

```

19. assistant (2026-06-22T17:42:56.116Z)

Now `\_execute\_processing`'s failed-result shape, to confirm parity, and `\_job\_to\_request`.

20. user (2026-06-22T17:42:57.310Z)

```

594     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
595         """The full hash ? validate ? segment ? report pipeline for one video.
596
597         This is the former synchronous /api/process body, unchanged in behavior, now run on
598         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
599         the sync endpoint used to return (UI compatibility); the per-video observability
600         report is still emitted in `finally:` and grafted as response["reports"].
601
602         `progress` is an optional callable(stage: str) used to surface coarse job progress.
603         Raises on unexpected internal errors ? the caller records those as a job failure
604         (after this function has already restored the pre-run segments, see below).
605         """
606         notify = progress or (lambda stage: None)
607
608         notify("hashing")
609         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
610         # (identity ?
611         # # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
612         # original
613         # # req.video_path (the source URI) is kept for the DB record + report; only the file-
614         # reading
615         # # consumers (hash / validators / segmenter) use the resolved local path.
616         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
617         "storage", None))
618         video_id = compute_video_id(local_video)
619         was_reingest = db_instance.get_video(video_id) is not None
620
621         # 1. Run pluggable validators (with-context variant surfaces ffmpeg_meta for the
622         # report)
623         notify("validating")
624         logger.info(f"Running validations for {req.video_path}...")
625         val_results, val_context = registry.run_all_with_context(local_video, global_config)
626         failures = [r for r in val_results if not r.passed]
627
628         # typed AppConfig: attribute access for the default segmenter (with safe fallback)

```

```

624     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
625     seg_name = req.segmenter or default_seg
626     segmenter_actual = None
627     segmentation_ran = False
628     response: Optional[Dict[str, Any]] = None
629     try:
630         if failures:
631             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
632             # status; it no longer destroys the video's prior good segments.
633             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
634             response = {
635                 "video_id": video_id,
636                 "status": "failed",
637                 "message": "Video failed validation checks.",
638                 "results": [r.model_dump() for r in val_results],
639             }
640             return response
641
642         # 2. Run segmenter & indexer
643         logger.info("[API] Video passed checks. Segmenting and indexing...")
644         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
645
646         segmenter_cls = segmenter_registry.get(seg_name)
647         if not segmenter_cls:
648             # Submit-time validation already 400s unknown names; this is the defensive
649             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
650
651             available = list(segmenter_registry.list_available().keys())
652             response = {
653                 "video_id": video_id,
654                 "status": "failed",
655                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
656                 "results": [r.model_dump() for r in val_results],
657                 "play_segments": [],
658             }
659             return response
660
661         logger.info(f"[API] Using segmenter: {seg_name}")
662         notify(f"segmenting ({seg_name})")
663         segmenter_actual = seg_name
664         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
665         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
666         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
667         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
668         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
669         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
670         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
671         # they're dropped on the cloud path in this slice ? a documented follow-up.)
672         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
673         if remote is not None:
674             response, segmentation_ran = remote # `ran` = did the cloud job actually
execute (report)
675             return response
676         segmenter = segmenter_cls(db_instance, global_config)

```

```

676         try:
677             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
678         except Exception:
679             # A raising segmenter must not leave half-written rows: restore the pre-run
680             # state (same destroy-after-confirm contract as the Failure branch), then
681             let
682                 # the job worker record the failure.
683                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
684                 raise
685             segmentation_ran = True
686
687         if isinstance(result, Failure):
688             logger.error(f"[API] Segmenter failed: {result.message}")
689             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
690             response = {
691                 "video_id": video_id,
692                 "status": "failed",
693                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
694                 "results": [r.model_dump() for r in val_results],
695                 "play_segments": [],
696             }
697             return response
698
699         segments = result.value if hasattr(result, "value") else result
700         notify("finalizing")
701         _finalize_reingest(video_id, segments, play_wm, cand_wm)
702
703         response = {
704             "video_id": video_id,

```

21. assistant (2026-06-22T17:43:03.157Z)

The failed-result shape from `\_execute\_processing` is `{"video\_id", "status": "failed", "message", ...}`. `\_execute\_compile`'s failed shape is `{"status": "failed", "message", "video\_id"}` ? matching the keys the SPA poll consumer reads (`result["status"]`, `result["message"]`). Parity confirmed.

Now let me check `\_job\_to\_request` (to confirm compile rows never hit it) and `\_run\_claimed\_job`'s surrounding error handling (the `mark\_job\_failed` path). Let me read the full `job\_worker.py` `\_run\_claimed\_job`.

22. user (2026-06-22T17:43:04.462Z)

```

75
76     def _run_claimed_job(job: Dict[str, Any]) -> None:
77         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
78         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
79
80         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
81         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
82         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
83         job self-scheduled and will be re-claimed when `next_poll_at` is due.
84         """
85         import backend.main as _main
86         db_instance = _main.db_instance
87         job_id = job["id"]
88         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
89         # original /api/process pipeline). Both record their terminal state the same way;
only process
90         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection

```

```

usage/segments).
91     is_compile = job.get("kind") == "compile"
92     try:
93         if is_compile:
94             result = _main._execute_compile(
95                 job, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
96         else:
97             req = _main._job_to_request(job)
98             result = _main._execute_processing(
99                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
100         if result.get("video_id"):
101             db_instance.set_job_video_id(job_id, result["video_id"])
102         db_instance.mark_job_done(job_id, result)
103         if not is_compile:
104             _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF; process
jobs only)
105             _maybe_run_flywheel(job, result) # M11 data-flywheel (process jobs only)
106     except _main.JobWaiting as w:
107         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
108         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
109         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
110     except Exception as e:
111         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
112         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
113
114
115     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
116         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
117         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
118         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
119         persist it.
120
121         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
122         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
123         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
124         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
125         With the placeholder pricebook the cost is 0 regardless.
126
127         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
128         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
129         import backend.main as _main
130
131         billing_cfg = getattr(_main.global_config, "billing", None)
132         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
133             return
134         try:

```

23. assistant (2026-06-22T17:43:11.352Z)

The dispatch is clean. For a process job (`kind` NULL ? `is\_compile` False), it runs `\_job\_to\_request` + `\_execute\_processing` + cost + flywheel exactly as before ? byte-identical. A stitch error inside `\_execute\_compile` is caught internally and returned as a `done`+failed result; if anything ELSE raises, the outer `except Exception` marks the job failed. Good.

Now let me verify a couple of potential issues. First ? `\_job\_to\_request`: does a process job require `video\_path` and does `create\_compile\_job` set the columns process expects? More importantly, let me check `\_add\_column\_idempotent` (migration idempotency), and whether the `jobs` table column order / NOT NULL constraints matter. Let me also check the `process\_video` flow re: `created\_at` default. Let me look at the jobs table CREATE statement and

```
`_add_column_idempotent`.
```

24. user (2026-06-22T17:43:13.489Z)

```
77:     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
78:         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
79-
80:         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
81-         --
237:             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
238-             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
239-             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE
240-             SET NULL
241-         )
242-         --
256:         CREATE TABLE IF NOT EXISTS jobs (
257-             id TEXT PRIMARY KEY,
258-             video_path TEXT NOT NULL,
259-             video_id TEXT,
260-             segmenter TEXT,
261-             player_pool TEXT,
262-             --
267:             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
268-             started_at TIMESTAMP,
269-             finished_at TIMESTAMP
270-         )
271-         --
282:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
283:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
284:         TIMESTAMP")
285:         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
286:         next_poll_at)")
287:         cursor.execute("PRAGMA user_version = 3")
288-         --
295:         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT")
296:         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
297:         user_id TEXT")
298:         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
299:         # of index churn while making the eventual per-user lookups cheap.
300:         cursor.execute(
301-             --
336:                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
337-             )
338:         """
339:         # One model version per user (NULLs compare distinct in SQLite, so the local
340-         --
374:         self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}")
375:         # Partial index keeps the NULL=no-cost fast path churn-free while making per-owner
376:         billing cheap.
377:         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
378:             "WHERE owner_id IS NOT NULL")
379-         --
408:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN locale TEXT")
409:         cursor.execute("PRAGMA user_version = 7")
410-         --
411:     def _migrate_to_v8(self, cursor):
412-         --
417:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
418:         cursor.execute("PRAGMA user_version = 8")
419-         --
420:     def _migrate_to_v9(self, cursor):
421-         --
426:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
427:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
```

```

428-         cursor.execute("PRAGMA user_version = 9")
429-
430-     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
431-         --
432-         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1").fetchone()
433-         if not row:
434-             return None
435-         cur.execute(

```

25. assistant (2026-06-22T17:43:18.135Z)

Migration confirmed: `kind`/`params` are nullable `TEXT` ADD COLUMNS via the idempotent helper (which I'll verify swallows the "duplicate column" error), `PRAGMA user\_version = 9` is set, and `SCHEMA\_VERSION = 9` matches. `jobs.video\_path` is `NOT NULL` ? and `create\_compile\_job` supplies `video["filepath"]` for it. Let me confirm `\_add\_column\_idempotent`'s body and the `\_execute\_write` error swallow behavior.

26. user (2026-06-22T17:43:19.244Z)

```

76     @staticmethod
77     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
78         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
79
80         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
81         transaction, so an interrupted v3/v4 block can leave the column added while
82         ``user_version`` stays un-bumped ? and the next open would re-run the same
83         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
84         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
85         Swallow ONLY that specific error so the ladder stays crash-safe, matching the re-
86         runnable
87         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
88         """
89         try:
90             cursor.execute(ddl)
91         except sqlite3.OperationalError as exc:
92             if "duplicate column name" not in str(exc).lower():
93                 raise
94
95     def _init_db(self):
96         """Initializes tables for videos, play_segments, and events."""
97         with self._connect() as conn:
98             cursor = conn.cursor()
99
100             self._create_base_tables(cursor)
101
102             # Versioned migrations live here. PRAGMA user_version is the schema
103             # contract ? bump it for every change and add an ordered `if version < N`
104             # block below. Tests assert the expected version, so accidental drift fails
105             CI.
106             version = cursor.execute("PRAGMA user_version").fetchone()[0]
107
108             # P2d upgrade-safety (no-op for a fresh DB or one already at
109             SCHEMA_VERSION):
110             # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
111             schema this
112             # binary can't reason about; opening it could corrupt data, so refuse
113             loudly.
114             if version > SCHEMA_VERSION:
115                 raise SchemaTooNewError(
116                     f"This database (schema v{version}) was created by a newer version
117                     of the app "
118                     f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
119                     point it at "

```

```

113             f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}")
114         # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
in place; a
115         #         failed migration could corrupt it, so snapshot it first (best-effort:
WARN, never
116         #         block the upgrade). version 0 = a brand-new DB ? nothing to back up.
117         if 0 < version < SCHEMA_VERSION:
118             self._backup_before_migrate(conn, version)
119
120         if version < 1:

```

27. assistant (2026-06-22T17:43:29.761Z)

Migration idempotency is solid. Now let me check the SPA caller(s). The breaking change is 200?202. I need to verify all in-repo callers of `/api/compile` are updated, including the bundled SPA (`backend/ui/`) and any scripts.

28. user (2026-06-22T17:43:31.645Z)

Found 26 files

```

C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_api_endpoints.py
C:\Users\avidu\Projects\bhi-asyncnccompile\backend\main.py
C:\Users\avidu\Projects\bhi-asyncnccompile\backend\infrastructure\database.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tools\generate_highlights.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_watermark_i18n.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_watermark.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_import_smoke.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_generate_highlights.py
C:\Users\avidu\Projects\bhi-asyncnccompile\scripts\run_process_and_stitch.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_api_cloud_dispatch.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_compiler.py
C:\Users\avidu\Projects\bhi-asyncnccompile\tests\test_config.py
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\archives\decisions\DECISIONS.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\UI_PROPOSAL.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\PLATFORM_ARCHITECTURE.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\PACKAGING_PLAN.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\COMPUTE_DECOUPLED_SERVING\README.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\CODE_MAP.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\CLOUD_GPU_DRYRUN_GUIDE.md
C:\Users\avidu\Projects\bhi-asyncnccompile\docs\BACKLOG.md
C:\Users\avidu\Projects\bhi-asyncnccompile\deploy\digitalocean\bootstrap.sh
C:\Users\avidu\Projects\bhi-asyncnccompile\backend\pipeline\stitcher\watermark_i18n.py
C:\Users\avidu\Projects\bhi-asyncnccompile\backend\pipeline\stitcher\compiler.py
C:\Users\avidu\Projects\bhi-asyncnccompile\backend\assets\fonts\README.md
C:\Users\avidu\Projects\bhi-asyncnccompile\backend\api\static\app.js
C:\Users\avidu\Projects\bhi-asyncnccompile\README.md

```

29. assistant (2026-06-22T17:43:35.375Z)

There's a bundled SPA at `backend/api/static/app.js`. This is the critical in-repo client. Let me check how it calls `/api/compile` ? does it expect a 200 sync response with `download\_url`, or does it poll? Let me also check `backend/ui/` (the wizard bundle mentioned in memory).

30. user (2026-06-22T17:43:37.030Z)

```

83-
84-         showLoader("Analyzing Video Stream", "Running structural diagnostics, video focus
checks, and relevance validators...");
85-
86-         try {
87-             // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
88-             const response = await fetch("/api/process", {
89-                 method: "POST",
90-                 headers: { "Content-Type": "application/json" },

```

```

91-         body: JSON.stringify({
92-             // Poll until the job reaches a terminal state (done | failed).
93-             let job;
94-             for (;;) {
95-                 const jr = await fetch(`/api/jobs/${jobId}`);
96-                 if (!jr.ok) {
97-                     const err = await jr.json();
98-                     throw new Error(err.detail || "Failed to poll job state.");
99-                 }
100-                job = await jr.json();
101-                if (job.status === "done" || job.status === "failed") break;
102-                showLoader("Analyzing Video Stream",
103-                    `Job ${job.status}${job.progress ? " ? " + job.progress : ""}?`);
104-                await new Promise(res => setTimeout(res, 3000));
105-            }
106-
107-            if (job.status === "failed") {
108-                throw new Error(job.error || "Processing job failed.");
109-            }
110-
111-            const data = job.result || {};
112-
113-            // Render validation list
114-            renderValidationResults(data.results);
115-
116-            if (data.status === "failed") {
117-                hideLoader();
118-                alert("Validation Failed! This video does not meet our quality or structural
119-                criteria. Please review the checklist.");
120-                renderEmptyRallies("Validation failed. Please load a valid badminton raw
121-                video.");
122-                compileBoard.style.display = "none";
123-
124-                const outputName = highlightOutputName.value.trim() || "highlights_compiled.mp4";
125-                showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg
126-                stitching demuxers...");
127-
128-                try {
129-                    const res = await fetch("/api/compile", {
130-                        method: "POST",
131-                        headers: { "Content-Type": "application/json" },
132-                        body: JSON.stringify({
133-                            video_id: currentVideoId,

```

31. assistant (2026-06-22T17:43:40.002Z)

This is the key finding. Let me read the full compile handler in `app.js` to see exactly how it consumes the response.

32. user (2026-06-22T17:43:40.314Z)

```

366         return;
367     }
368
369     const outputName = highlightOutputName.value.trim() ||
370     "highlights_compiled.mp4";
371     showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg
372     stitching demuxers...");
373
374     try {
375         const res = await fetch("/api/compile", {
376             method: "POST",

```



```

375         headers: { "Content-Type": "application/json" },
376         body: JSON.stringify({
377             video_id: currentVideoId,
378             // API field is segment_ids (the old rally_ids never matched the
request model)
379             segment_ids: Array.from(selectedRallyIds),
380             output_filename: outputName
381         })
382     });
383
384     if (!res.ok) {
385         const err = await res.json();
386         throw new Error(err.detail || "Concatenation failed.");
387     }
388
389     await res.json();
390     // B3 (PR-2): offer the reel via the streaming download endpoint instead of
391     // alerting a server-local path the browser can't reach.
392     const dlUrl =
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
393     let dl = document.getElementById("highlight-download-link");
394     if (!dl) {
395         dl = document.createElement("a");
396         dl.id = "highlight-download-link";
397         dl.style.cssText = "display:block;margin-top:10px;font-weight:bold;";
398         compileBoard.appendChild(dl);
399     }
400     dl.href = dlUrl;
401     dl.setAttribute("download", outputName);
402     dl.textContent = `? Download ${outputName}`;
403     alert(`Highlight compilation completed! Use the download link to save
${outputName}.`);
404
405     } catch (error) {
406         alert(`Stitching compilation failed: ${error.message}`);
407     } finally {
408         hideLoader();
409     }
410     });
411 });
412

```

33. assistant (2026-06-22T17:43:47.136Z)

This is a real finding. The bundled SPA (`backend/api/static/app.js`) was **not** updated to poll. It treats `/api/compile` as synchronous: it fires the POST, checks `res.ok` (202 IS ok), calls `await res.json()` (discards the body ? which is now the `{job\_id, status:'queued', poll}` envelope, not the reel result), then **immediately** builds the download link and pops "compilation completed!". The reel hasn't been built yet ? the stitch is still running on the worker.

Let me confirm this `app.js` is the served/bundled SPA (not dead code) and whether `backend/ui/` has a separate, updated copy. Let me also check the khelsutra sibling SPA isn't the only client (the task says "ALL in-repo callers"). Let me verify which static dir is served and whether there's a `backend/ui/`.

34. user (2026-06-22T17:43:50.230Z)

```

=== app.js git status (modified in this branch?) ===
5eaba74 feat(api): chunked upload + highlight download (B2+B3) - static UI compile fix +
download link (#99)
=== is app.js touched in this diff? ===
NOT in diff
=== static mount in main.py ===
40:from fastapi.staticfiles import StaticFiles

```

```

185:# Serves static frontend files directly (now under api/ per approved structure).
187:# arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
189:_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
196:app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
198:# P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
207:     app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
220:     """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
=== does backend/ui exist? ===
THIRD_PARTY_NOTICES.md
assets
index.html
ui_version.json

```

35. assistant (2026-06-22T17:43:54.165Z)

So there are TWO bundled frontends: `backend/api/static/app.js` (the legacy static UI, served at `/static`) and `backend/ui/` (the vendored KhelSutra SPA, served single-origin). Neither is touched in this diff. Let me check the `backend/ui/` bundled SPA (the primary alpha client per memory) for how it calls `/api/compile`. It's minified, so let me grep it.

36. user (2026-06-22T17:43:56.402Z)

```

=== ui/index.html refs to compile ===
=== assets dir ===
index-BcE6Y0gS.css
index-GSXqL6iY.js
noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
noto-sans-kannada-kannada-wght-normal-DuWB-P8d.woff2
noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
=== grep compile in ui assets ===

```

37. user (2026-06-22T17:44:01.115Z)

```

=== compile string in minified js ===
compile-result
compile-result
compile-error
compile
=== /api/ endpoints referenced in the bundled SPA ===

```

38. assistant (2026-06-22T17:44:04.052Z)

The endpoints are likely built via template strings/variables. Let me look at the context around the `compile` token and how it polls.

39. user (2026-06-22T17:44:05.772Z)

```

=== context around 'compile' token (string) ===
s:1}){const{t:r}=wt();if(i.status==="idle")return null;if(i.status==="building")return
T.jsx("section",{data-testid:"compile-result",className:"mt-6 rounded-xl border border-
black/10 bg-paper p-4","aria-live":"polite",children:[T.jsx(Ss,{tone:"info",icon:T.jsx(vo,{}),la
bel:r("result.building"))},T.jsx("p",{className:"mt-2
t-ink/60",children:r("result.buildingReassure"))})]);const u=i.status==="error";return
T.jsx("section",{data-testid:"compile-result","aria-live":"polite",className:"mt-6 rounded-xl
border p-4 "+(u?"border-red-200 bg-red-50":"border-green/30 bg-
green/5"),children:[T.jsx("div",{className:"flex items-start justify-between ga
en":"true",children:"?")},r("result.share"))])]);T.jsx("p",{className:"mt-2 text-sm text-
red-800","data-testid":"compile-error",children:i.error}))}}function qt(){const

```

```
i=globalThis.crypto;return i&&typeof i.randomUUID=="function"?i.randomUUID():`cid_${Date.now().t
oString(36)}_${Math.floor(Math.random()*1e9).toString(
y",{method:"POST",body:JSON.stringify({video_id:i}),cid:l}).play_segments??[]}async function
_1(i,l,r,u,c){return ca("/compile",{method:"POST",body:JSON.stringify({video_id:i,segment_ids:l,
output_filename:r,locale:c}),cid:u})}function
L1(i,l){return`${To}/highlights/${encodeURIComponent(i)}/${encodeURIComponent(l)}`}async fu
```

40. assistant (2026-06-22T17:44:10.142Z)

The bundled khelsutra SPA's compile call is `\_1(...)` which does `ca("/compile", {method:"POST", ...})` and returns the parsed JSON directly. There's a "building" status, so let me check whether this SPA polls after the compile POST, or treats the POST response as the final result. Let me trace what consumes `\_1`'s return and whether there's a job-poll.

41. user (2026-06-22T17:44:12.964Z)

```
=== does the SPA poll /jobs/ ? ===
/jobs/${encodeURIComponent(i
=== context: how _1 (compile) return is used + 'building'/'ready' transitions ===
ST",body:JSON.stringify({video_id:i}),cid:l}).play_segments??[]}async function
_1(i,l,r,u,c){return ca("/compile",{method:"POST",body:JSON.stringify({video_id:i,segment_ids:l,
output_filename:r,locale:c}),cid:u})}function
L1(i,l){return`${To}/highlights/${encodeURIComponent(i)}/${encodeURIComponent(l)}`}async
function j1(i){return ca("/confi
V.totalSec),below_floor:(Q==null?void 0:Q.atRisk.length)?0});try{const P=await _1(r,ie,q,z,l.re
solvedLanguage??l.language),ee=L1(r,q);L({status:"done",url:ee,filename:q}),Se("info","build.don
e",{cid:z,video_id:r,filename:q,filepath:P.filepath,url:ee})}catch(P){const ee=P instanceof
Yn?P.status:void 0,J=Zl(P);L({status:"error",error:So(i,J)}}
```

42. assistant (2026-06-22T17:44:22.448Z)

This confirms the finding for the khelsutra SPA too. Tracing `P=await \_1(...)` then immediately `L({status:"done",url:ee,...})` ? it sets status to **done** right after the POST resolves, building the download URL from the filename. It does NOT poll `/api/jobs/{id}`. The `/jobs/` reference exists but is for the `/api/process` flow, not compile.

So **both** bundled in-repo SPAs treat `/api/compile` synchronously and will break: they mark the reel "done"/offer the download link immediately on the 202 response, before the worker has actually built the reel. The download will 404 (or serve a stale/partial file) because the stitch is still running.

Let me confirm the download endpoint behavior ? what happens if the SPA hits `/api/highlights/{vid}/{name}` before the worker finishes (404, or a partial-file serve). And verify there are no other server-side callers (scripts) that break.

43. user (2026-06-22T17:44:24.139Z)

```
432:   Mirrors /api/highlights' safety (bare-name + containment checks)."""
433:   try:
434:       name = safe_output_filename(filename)
435:   except ValueError as e:
436:       raise HTTPException(status_code=400, detail=str(e)) from e
437:   base = _reels_dir()
438:   path = os.path.join(base, name)
439:   try:
440:       path = resolve_under_root(path, base)
441:   except ValueError as e:
442:       raise HTTPException(status_code=400, detail=str(e)) from e
443:   if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
444:       raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
--
834:@app.get("/api/highlights/{video_id}/{filename}")
835-def download_highlight(video_id: str, filename: str):
836:    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
```

```
837- (which only writes into output_dir; the browser previously got back an unreachable
838- server-local path)."""
839- try:
```

[Showing results with pagination = limit: 20]

44. assistant (2026-06-22T17:44:28.854Z)

Confirmed: the download endpoint returns 404 if the reel file isn't on disk yet. So when the SPA offers the download link immediately on the 202 response, the user clicks it and gets a 404 until the worker finishes ? exactly the broken UX, and worse than before (before, the 200 came only after the reel existed).

Now let me verify the scripts callers (`tools/generate\_highlights.py`, `scripts/run\_process\_and\_stitch.py`) ? do they hit the HTTP `/api/compile` endpoint, or call `VideoCompiler` directly? If they call the endpoint, they break too.

45. user (2026-06-22T17:44:30.994Z)

```
=== generate_highlights.py: HTTP or direct? ===
13: duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
41:from backend.pipeline.stitcher.compiler import VideoCompiler
219:         VideoCompiler(
224:         ).compile_highlights(stitch_src, pairs, reel_name)
=== run_process_and_stitch.py ===
18:from backend.pipeline.stitcher.compiler import VideoCompiler
158:     compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding,
161:         out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
```

46. assistant (2026-06-22T17:44:40.357Z)

Good ? both scripts call `VideoCompiler` directly (not the HTTP endpoint), so they're unaffected. The only broken in-repo callers are the two bundled SPAs.

Now let me double-check one more subtle thing the prompt flagged: the `created\_at` column. `create\_compile\_job`'s INSERT doesn't list `created\_at`, and the table defaults it to `CURRENT\_TIMESTAMP` ? and `claim\_due\_job` orders by `COALESCE(next\_poll\_at, created\_at)`. So compile jobs do get a `created\_at` and order correctly. Good.

Let me also verify there's no issue with `status\_code=202` being set both on the decorator AND the `JSONResponse` (harmless, but let me confirm it's consistent with `/api/process` ? it is, line 723 does the same). And let me confirm the test suite for the worker-dispatch path runs the inline executor properly. Let me run the directly affected tests to confirm the server-side change is sound (the SPA bug won't be caught by tests since tests poll explicitly).

47. user (2026-06-22T17:45:02.137Z)

```
..... [ 67%]
..... [100%]
107 passed in 18.46s
```

48. assistant (2026-06-22T17:45:36.980Z)

All 107 touched tests pass. The server-side change is internally sound and well-tested. The gap is purely that the tests poll explicitly (mirroring the *intended* new client contract) while the actual bundled SPA clients were never updated to poll.

Let me do one final verification of the claim in the prompt about `\_resolve\_compile` being safe to run twice and the segment-deletion-between-submit-and-run case. `\_resolve\_compile` is pure-read (get_video + query_play_segments + filtering) with no side effects, so running it twice is safe. If segments are deleted between submit and run, the worker's `\_resolve\_compile` raises `\_CompileError(400, "No matching play segments...")`, caught in `\_execute\_compile` ? returns `{ "status": "failed", ... }` ? `mark\_job\_done`. Clean. Confirmed from the code already read.

Let me also confirm ``safe_output_filename`` is enforced at the worker too (it is ? ``_resolve_compile`` calls it, and the worker calls ``_resolve_compile``). And ``owner_id`` is captured at submit on the compile job (line 210-212, passes ``owner_id=owner_id``). Both verified.

One last check ? is the ``owner_id`` actually used to scope anything on the compile path, or is it purely captured for the ledger? It's captured but compile skips cost/flywheel. That's fine for default-OFF single-tenant; cross-tenant isolation on compile would be a pre-existing concern, not introduced here (the resolve is by ``video_id`` with no owner filter, same as the old sync endpoint). Not a regression.

Findings

I reviewed the full diff plus the surrounding code (``main.py``, ``database.py``, ``job_worker.py``, both bundled SPAs, the script callers, and the download endpoint), and ran the 107 touched tests (all pass). The **server-side async refactor is correct and well-tested**. There is **one real bug**, in the clients.

MAJOR ? Bundled SPAs were not updated to poll; they break on the 202 and offer a download link before the reel exists

- **Backend** ``backend/api/static/app.js:373-403`` (legacy static UI, served at ``/static``) and ``backend/ui/assets/index-GSxqL6iY.js`` (the vendored KhelSutra alpha SPA, served single-origin) ? neither is touched in this diff.
- Both treat ``/api/compile`` as **synchronous**. They POST, see ``res.ok`` (202 passes), discard the new envelope body (``await res.json()`` / ``P=await _1(...)``), and **immediately** mark the compile "done" and build the download link from ``/api/highlights/{video_id}/{output_filename}`` ? while the ffmpeg stitch is still running on the worker.
- Concretely: in ``app.js``, line 389 ``await res.json()`` throws away ``{job_id, status:"queued", poll}``, then lines 392-403 build the link and ``alert("compilation completed!")``. In the KhelSutra bundle, ``P=await _1(...)`` resolves on the 202 and the next statement does ``L({status:"done", url:ee, ...})``.
- **Consequence:** the download endpoint (``backend/main.py:834`` ``download_highlight``) returns **404** when the file isn't on disk yet (``backend/main.py:443``). So the user is shown a "done"/download link that 404s until the stitch finishes. This is a worse UX than before the change (the old 200 only arrived once the reel existed). The 524-on-slow-stitch problem is fixed at the engine but **not** delivered to the user, because no client polls the new ``poll`` URL. This is exactly the "needs a paired SPA poll change" the prompt anticipated ? and it is missing for both in-repo SPAs.
- **Fix:** update both SPA compile handlers to mirror their own existing ``/api/process`` poll loop ? on 202, read ``job_id``, poll ``GET /api/jobs/{job_id}`` until ``status`` is ``done``/``failed``, then read ``result.download_url`` (or ``result.status === "failed"`` ? show ``result.message``). ``app.js`` already has the exact pattern at lines 105-134; the KhelSutra bundle already polls ``/jobs/${...}`` for process. (For the vendored bundle this means a rebuild of the upstream khelsutra SPA, not a hand-edit of the minified asset ? flag the source-repo change.)

Verified clean (no issue) ? everything else the prompt asked to probe

- **Default-OFF / process unchanged:** ``kind`` NULL ? ``is_compile`` False ? ``_job_to_request`` + ``_execute_processing`` + ``_maybe_record_job_cost`` + ``_maybe_run_flywheel`` run byte-identically (``job_worker.py:91-105``). Cost/flywheel are correctly gated off for compile only.
- **Migration:** ``_migrate_to_v9`` (``database.py:420-428``) is additive (two nullable ``TEXT`` ADD COLUMNS via ``_add_column_idempotent``, which swallows only "duplicate column name"), sets ``PRAGMA user_version = 9``, and ``SCHEMA_VERSION`` is bumped to 9. Pre-v9 rows have ``kind`` NULL ? behave as process. No NOT NULL trap: the ``jobs.video_path NOT NULL`` column is satisfied ? ``create_compile_job`` stores ``video["filepath"]`` there.
- **Worker receives params:** ``claim_due_job`` returns ``get_job(id)`` = ``SELECT *``, so ``kind`` and ``params`` reach the worker. ``get_job`` (``database.py:737-747``) deserializes only ``result``/``player_pool``/``policy`` ? ``params`` stays a **raw JSON string**, and ``_execute_compile`` correctly does ``json.loads(job.get("params") or "{}")`` (``main.py:232``). No double-decode.
- **Submit-vs-worker resolution:** ``_resolve_compile`` is pure-read (no side effects), safe to run twice. Segments deleted between submit and run ? worker raises ``_CompileError(400)`` ? caught ? ``{status:"failed"}`` done-job, no crash. ffmpeg raising ? caught at ``main.py:271-273`` ? ``{status:"failed"}`` done-job.
- **Error-shape parity:** ``_execute_compile``'s failed result ``{"status":"failed","message",...,"video_id"}`` matches ``_execute_processing``'s failed shape; the

job-poll consumer reads `result.status`/`result.message` identically.

- **Security:** `safe\_output\_filename` enforced at submit AND worker (both via `\_resolve\_compile`). `owner\_id` captured on the compile job (`main.py:199-213`) exactly like `/api/process`; missing/spoofed edge-auth identity ? 401 before queuing.
- **Tests:** all in-repo HTTP/script callers other than the two SPAs are unaffected (`tools/generate\_highlights.py`, `scripts/run\_process\_and\_stitch.py` call `VideoCompiler` directly). The updated test suite asserts the new 202+poll contract and passes (107/107).

Net: the engine change is solid; the PR is **incomplete** because the breaking 200?202 contract change ships without updating either bundled client, so the user-visible compile flow regresses to a download-link-404. That client fix should land with (or block) this PR.